

INFORME TÉCNICO

Investigación e Implementación de API Externa

SendGrid Email API

Integración en Texticode — Sistema de Gestión Textil

Aprendiz Andres Felipe Lancheros Camilo Andres Tibambre Kevin Andres Castro Kevin Stevan Wagner	Instructor Ing. Edwin Marín
Fecha Mayo 2026	Proyecto Texticode — ADSO 2026

★ Implementación real verificada con evidencias de prueba ★

1. API SELECCIONADA: SENDGRID EMAIL API

1.1 Justificación de la elección

SendGrid (Twilio) es la plataforma de envío de correo transaccional más utilizada en el mundo para aplicaciones web. Para el proyecto Texticode — sistema de gestión de producción textil — la integración de SendGrid es completamente natural porque el sistema ya maneja clientes, órdenes, operarios y tareas de producción, pero carecía de un mecanismo de notificación automática.

Necesidad en Texticode	Solución implementada con SendGrid
Usuario olvida su contraseña	Email automático con enlace seguro de recuperación (expira en 1 hora)
Admin asigna tarea a operario	Email al operario con nombre de tarea, orden #, prioridad y fecha límite
Operario marca pedido como completado	Email al cliente informando que su orden está lista para entrega
Clientes necesitan saber cuándo su orden cambia	Email automático al actualizar el estado de la orden
Registro de comunicaciones	Historial de emails enviados por orden y por usuario

1.2 Problema que resuelve la API

Texticode no tenía mecanismo de notificación a clientes ni operarios. Todas las actualizaciones de estado se hacían manualmente y el cliente no sabía cuándo su pedido estaba listo.

SendGrid resolvió tres flujos reales en el sistema:

- Recuperación de contraseña: el usuario solicita un reseteo y recibe un enlace con token seguro que expira en 1 hora.
- Notificación de tarea asignada: cuando el admin asigna una tarea de producción a un operario, este recibe un email con todos los detalles.
- Orden lista para entrega: cuando el operario marca un pedido como completado, el cliente recibe confirmación automática por correo.

2. INVESTIGACIÓN TÉCNICA DE LA API

2.1 Documentación oficial



URL de documentación:

<https://docs.sendgrid.com/api-reference> SDK Node.js disponible en npm: @sendgrid/mail

2.2 Métodos HTTP y endpoints clave

Método	Endpoint	Uso en Texticode	Respuesta
POST	/v3/mail/send	Enviar notificaciones (tarea, contraseña, orden lista)	202 Accepted
GET	/v3/stats	Ver estadísticas de emails enviados por día	opens, clicks, bounces
GET	/v3/messages	Consultar historial de correos enviados	messages[] con estado
DELETE	/v3/suppression/bounces/:email	Limpiar rebotes para reactivar envíos	204 No Content

2.3 Autenticación

SendGrid usa API Key como único método de autenticación en su versión REST moderna:

- API Key en el header HTTP: Authorization: Bearer SG.xxxxx
- El SDK de Node.js configura esto automáticamente con `sgMail.setApiKey(process.env.SENDGRID_API_KEY)`
- Las API Keys se pueden restringir por scopes (solo envío, solo estadísticas, etc.)
- Se crean en el panel de SendGrid bajo Settings → API Keys

```
// Autenticación con el SDK oficial de SendGrid (Node.js)
// services/emailService.js
import sgMail from '@sendgrid/mail';

// API Key desde .env - nunca hardcodeada
sgMail.setApiKey(process.env.SENDGRID_API_KEY);

// .env (texticode-api/)
// SENDGRID_API_KEY=SG.qdwB9Ag5S76XC8rgjtqr1g...
// SENDGRID_FROM_EMAIL=stevanwagner1@gmail.com
```

2.4 Límites de uso y políticas

Aspecto	Detalle
Plan gratuito	100 emails/día de forma permanente. Sin tarjeta de crédito. Ideal para desarrollo y pruebas.
Plan Essentials	USD 19.95/mes para 50.000 emails/mes. Suficiente para empresa textil PYME.
Rate limiting	600 requests/minuto en todos los planes. Sin restricción para uso normal.
Tiempo de entrega	Menos de 1 segundo de latencia promedio para entrega al servidor SMTP destino.
Adjuntos	Máximo 30 MB por email en total. Suficiente para PDFs de comprobantes.

3. IMPLEMENTACIÓN TÉCNICA REAL EN TEXTICODE

3.1 Estructura del proyecto con integración SendGrid

A continuación se muestra la estructura real del repositorio Texticode-Vue con los archivos nuevos y modificados marcados con ★:



Archivo	Rol en la integración SendGrid
services/emailService.js	Cerebro del sistema de emails. Contiene 3 funciones: enviar link de recuperación de contraseña, notificar al operario una tarea nueva, y avisar al cliente que su orden está lista.
notificaciones.js	Define los endpoints REST que llama el frontend: POST /tarea-asignada (avisa al operario) y POST /orden-lista (avisa al cliente). Cada uno llama a emailService.
auth.route.js	Maneja el olvido de contraseña: recibe el email del usuario, genera un token seguro con expiración de 1 hora, lo guarda en BD y dispara el email de recuperación.
index.js (api)	Punto de entrada del backend. Registra las rutas de notificaciones (/notificaciones) y autenticación (/auth) para que estén disponibles en el servidor Express.
composables/useNotificaciones.js	Capa de comunicación del frontend. Expone funciones reutilizables que los componentes Vue llaman para disparar emails, sin repetir lógica fetch en cada vista.
views/admin/GestionProduccion.vue	Al asignar una tarea a un operario, llama a useNotificaciones para enviarle el email. También actualiza el estado de la orden cuando el operario la completa.
views/LoginView.vue	Incluye el flujo de recuperación de contraseña: el usuario ingresa su email y el sistema llama a auth.route.js, que envía el enlace de reseteo por SendGrid.
views/operario/ReportarAvances.vue	El operario marca la tarea como completada aquí. Eso dispara la llamada a useNotificaciones, que avisa al cliente por email que su orden está lista para entrega.



FUNCIONALIDAD 1: Recuperación de contraseña

3.2 Email de recuperación de contraseña

Cuando un usuario olvida su contraseña, puede solicitar un email de recuperación desde el formulario de login. El sistema genera un token seguro, lo guarda en la base de datos con expiración de 1 hora, y envía un email con el enlace de cambio de contraseña.

Flujo completo:

- Usuario hace clic en '¿Olvidaste tu contraseña?' en LoginView.vue
- Frontend llama al endpoint POST /auth/forgot-password con el email
- auth.route.js genera un token único, lo guarda en BD con expiración 1h
- Se llama a emailService.sendPasswordReset(email, token)
- SendGrid envía el email con el botón 'Cambiar contraseña' que incluye el token
- El enlace expira en 1 hora por seguridad

```
// texticode-api/services/emailService.js
import sgMail from '@sendgrid/mail';
sgMail.setApiKey(process.env.SENDGRID_API_KEY);

export async function sendPasswordReset(email, nombre, resetToken) {
  const resetUrl = `http://localhost:5173/reset-password?token=${resetToken}`;
  const msg = {
    to: email,
    from: process.env.SENDGRID_FROM_EMAIL,
    subject: 'Texticode - Cambio de contraseña',
    html: `
      <div style='background:#1B3A5C;padding:20px;text-align:center'>
        <h1 style='color:white'>Texticode</h1>
        <p style='color:#AAD4F0'>Sistema de gestión textil</p>
      </div>
      <div style='padding:30px;text-align:center'>
        <h2>🔒 Cambio de contraseña</h2>
        <p>Hola <strong>${nombre}</strong>, recibimos una solicitud para cambiar tu
        contraseña.</p>
        <a href='${resetUrl}' style='background:#1B3A5C;color:white;padding:12px
        24px;
        border-radius:6px;text-decoration:none'>Cambiar contraseña</a>
        <p
        style='background:#FFF3CD;padding:10px;border-radius:4px;margin-top:20px'>
        ⌚ Este enlace expira en <strong>1 hora</strong>.
```

```

        </p>
      </div>
    ,
  ];
  await sgMail.send(msg);
}

```



FUNCIONALIDAD 2: Notificación de tarea asignada al operario

3.3 Email de nueva tarea de producción asignada

Cuando un administrador asigna una tarea de producción a un operario desde GestionProduccion.vue, el operario recibe automáticamente un email con todos los detalles de la tarea: nombre, número de orden, prioridad y fecha límite.

Flujo completo:

- Admin asigna tarea en GestionProduccion.vue
- El frontend llama al composible useNotificaciones.js con los datos de la tarea
- useNotificaciones hace POST al endpoint /notificaciones/tarea-asignada
- El backend llama emailService.sendTareaAsignada(operarioEmail, datos)
- Operario recibe email con: nombre de tarea, orden #, prioridad (Alta/Media/Baja) y fecha límite

```

// texticode-api/services/emailService.js
export async function sendTareaAsignada(email, nombre, tarea) {
  const prioridadColor = {
    'Alta': '#DC3545', 'Media': '#FFC107', 'Baja': '#28A745'
  }[tarea.prioridad] || '#6C757D';

  const msg = {
    to: email,
    from: process.env.SENDGRID_FROM_EMAIL,
    subject: `Texticode - Nueva tarea asignada: ${tarea.nombre}`,
    html: `
      <div style='background:#5B3FA6;padding:20px;text-align:center'>
        <p style='color:white;letter-spacing:2px'>TEXTICODE - SISTEMA DE GESTIÓN
        TEXTIL</p>
      </div>
      <div style='padding:24px'>

```



```

    <h2>🔧 Nueva tarea asignada</h2>
    <p>Hola <strong>${nombre}</strong>, se te ha asignado una nueva tarea de
    producción.</p>
    <table style='width:100%;border:1px solid #ddd;border-radius:8px'>
      <tr><td><strong>TAREA</strong></td></tr>
      <tr><td>${tarea.nombre} - ${tarea.producto}</td></tr>
      <tr>
        <td>ORDEN: <strong>#${tarea.ordenId}</strong></td>
        <td>PRIORIDAD: <span style='background:${prioridadColor};color:white;
          padding:4px 10px;border-radius:12px'>${tarea.prioridad}</span></td>
      </tr>
      <tr><td>FECHA LÍMITE: ${tarea.fechaLimite}</td></tr>
    </table>
  </div>
  ,
  };
  await sgMail.send(msg);
}

```

✓ FUNCIONALIDAD 3: Orden lista para entrega (notificación al cliente)

3.4 Email de orden lista para entrega al cliente

Cuando un operario marca un pedido como completado en el sistema, el cliente recibe automáticamente un email informando que su orden está lista para ser recogida o enviada. El email incluye el número de orden, el estado 'Lista para entrega' y los productos incluidos.

Flujo completo:

- Operario marca pedido como completado en TareasAsignadas.vue
- El frontend llama al endpoint PATCH /ordenes/:id con el nuevo estado
- El backend actualiza el estado en base de datos
- Se llama internamente a emailService.sendOrdenLista(clienteEmail, datos)
- Cliente recibe email con número de orden, estado 'Lista para entrega' y productos

```
// texticode-api/services/emailService.js
export async function sendOrdenLista(email, clienteNombre, orden) {
  const msg = {
    to: email,
    from: process.env.SENDGRID_FROM_EMAIL,
    subject: `Texticode - Orden #${orden.id}: Lista para entrega`,
    html: `
      <div style='background:#1A7A4A;padding:20px;text-align:center'>
        <p style='color:white;letter-spacing:2px'>TEXTICODE - SISTEMA DE GESTIÓN
        TEXTIL</p>
      </div>
      <div style='padding:24px'>
        <h2>✓ Tu orden ha sido actualizada</h2>
        <p>Hola <strong>${clienteNombre}</strong>,
        te informamos que el estado de tu orden ha cambiado.</p>
        <table style='width:100%;border:1px solid
        #ddd;border-radius:8px;padding:16px'>
          <tr>
            <td><small>ORDEN</small><br><strong>#${orden.id}</strong></td>
            <td><small>ESTADO ACTUAL</small><br>
            <span style='background:#1A7A4A;color:white;padding:6px 14px;
            border-radius:20px'>Lista para entrega</span>
          </td>
        </tr>
        <tr><td colspan='2'>
```

```

        <small>PRODUCTOS</small><br>${orden.productos}
      </td></tr>
    </table>
    <p>Si tienes alguna pregunta sobre tu pedido, puedes responder directamente
a este correo.</p>
  </div>
  <div style='background:#F5F5F5;padding:16px;text-align:center'>
    <small>Este es un correo automático generado por Texticode.<br>
    Si tienes dudas, contacta al administrador del sistema.</small>
  </div>
  ,
};
await sgMail.send(msg);
}

```

3.5 Endpoint central de notificaciones

El archivo notificaciones.js expone el endpoint que recibe todos los eventos y los enruta al servicio correspondiente:

```

// texticode-api/notificaciones.js
import { Router } from 'express';
import { sendTareaAsignada, sendOrdenLista } from '../services/emailService.js';
const router = Router();

// POST /notificaciones/tarea-asignada
router.post('/tarea-asignada', async (req, res) => {
  const { email, nombre, tarea } = req.body;
  try {
    await sendTareaAsignada(email, nombre, tarea);
    res.json({ success: true, mensaje: 'Notificación enviada al operario.' });
  } catch (error) {
    res.status(500).json({ success: false, error: error.message });
  }
});

// POST /notificaciones/orden-lista
router.post('/orden-lista', async (req, res) => {
  const { email, clienteNombre, orden } = req.body;
  try {
    await sendOrdenLista(email, clienteNombre, orden);
    res.json({ success: true, mensaje: 'Notificación enviada al cliente.' });
  }
});

```

```
    } catch (error) {
      res.status(500).json({ success: false, error: error.message });
    }
  });

export default router;
```

3.6 Composable Vue — useNotificaciones.js

En el frontend, el composable centraliza todas las llamadas HTTP al backend de notificaciones, manteniendo el código limpio y reutilizable:

```
// texticode-vue/src/composables/useNotificaciones.js
export function useNotificaciones() {
  const API = 'http://localhost:3001';

  async function notificarTareaAsignada(operario, tarea) {
    await fetch(`${API}/notificaciones/tarea-asignada`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        email: operario.email,
        nombre: operario.nombre,
        tarea
      })
    });
  }

  async function notificarOrdenLista(cliente, orden) {
    await fetch(`${API}/notificaciones/orden-lista`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        email: cliente.email,
        clienteNombre: cliente.nombre,
        orden
      })
    });
  }

  return { notificarTareaAsignada, notificarOrdenLista };
}
```

4. SEGURIDAD Y BUENAS PRÁCTICAS

4.1 Protección de credenciales

- La API Key se almacena exclusivamente en el archivo .env local del backend (texticode-api/.env)
- .env se agrega a .gitignore — nunca se sube al repositorio
- En producción se usan variables de entorno del servidor (Render, Railway, etc.)
- La API Key se crea con scope mínimo necesario (solo 'Mail Send' para el backend)
- Los tokens de recuperación de contraseña se almacenan hasheados y con fecha de expiración

```
# texticode-api/.env - archivo local, NUNCA subir al repositorio
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=
DB_NAME=texticode
PORT=3001
SENDGRID_API_KEY=SG.qdwB9Ag5S76XC8rgjtqrlg... ← nunca en el repo
SENDGRID_FROM_EMAIL=stevanwagner1@gmail.com

# texticode-api/.gitignore
.env
node_modules/
```

4.2 Riesgos de seguridad identificados

Riesgo	Mitigación aplicada
Exposición de la API Key	Uso de .env + .gitignore. La key nunca se expone en el frontend ni en el repositorio.
Token de reseteo interceptado	Token expira en 1 hora y se invalida tras su primer uso. Se guarda hasheado en BD.
Email spoofing	Verificar el dominio remitente en SendGrid (DNS DKIM/SPF). Sin verificación no se puede enviar.
Spam / abuso del endpoint	Validar que el email sea válido antes de llamar la API. Rate limiting en Express.
Rebotes acumulados	Monitorear endpoint GET /estadisticas. Si bounces > 5%, limpiar lista.

5. PREGUNTAS DE INVESTIGACIÓN

P1. ¿Qué problema resuelve la API en Texticode?

Texticode gestiona órdenes, producción y clientes, pero no tenía comunicación automática. El admin actualizaba estados manualmente y el cliente no se enteraba hasta que llamaba o escribía. SendGrid resolvió tres flujos reales: (1) recuperación de contraseña con token seguro y enlace temporal, (2) notificación al operario cuando se le asigna una tarea de producción con prioridad y fecha límite, y (3) notificación al cliente cuando su orden está lista para entrega. Esto elimina la comunicación manual y cierra el ciclo de información del sistema.

P2. Métodos HTTP y datos que retorna

POST /v3/mail/send: recibe un objeto JSON con to, from, subject, html y opcionalmente attachments. Retorna HTTP 202 Accepted si el email fue aceptado para entrega (sin cuerpo de respuesta). GET /v3/stats: retorna un arreglo con métricas diarias: requests, delivered, opens, clicks, bounces, spam_reports. No retorna datos personales de destinatarios.

P3. ¿Cómo se autentica?

Mediante API Key en el header HTTP Authorization: Bearer SG.xxxx. La clave se genera en el panel de SendGrid con scopes específicos (mínimo 'Mail Send' para el backend de Texticode). El SDK @sendgrid/mail abstrae este header completamente: basta con llamar sgMail.setApiKey() una sola vez al iniciar el servidor, en services/emailService.js.

P4. Límites gratuitos y políticas de uso

El plan gratuito incluye 100 emails por día de forma permanente, sin vencimiento ni tarjeta de crédito. Para una empresa textil PYME con 20-30 órdenes mensuales y un equipo pequeño de operarios, el plan gratuito es completamente suficiente durante el desarrollo y las pruebas académicas. Si el volumen crece, el plan Essentials ofrece 50.000 emails por mes por USD 19.95.

P5. Riesgos de seguridad

El principal riesgo es la exposición de la API Key en el repositorio, lo que permitiría a terceros enviar emails como si fueran Texticode. Un segundo riesgo son los tokens de recuperación de contraseña interceptados si no tienen expiración. Un tercer riesgo es enviar correos sin verificar el dominio remitente, lo que genera que los emails lleguen a spam. Los tres se mitigan con .env + .gitignore, tokens con expiración de 1 hora, y verificación DNS del dominio en el panel de SendGrid.

P6. Mejora implementada en Texticode

La integración se realizó directamente en el sistema real. GestionProduccion.vue fue modificado para llamar al composable useNotificaciones.js al asignar tareas. La ruta auth.route.js maneja el flujo completo de recuperación de contraseña. El servicio emailService.js centraliza las tres funciones de envío, y notificaciones.js expone los endpoints REST que el frontend consume. Toda la lógica de email es transparente para el usuario final.

6. EVIDENCIAS DE PRUEBA

6.1 Pruebas locales — 26/26 pasadas

Se ejecutaron las pruebas unitarias del módulo de email con Jest. Resultado: 26 pruebas pasadas, 0 fallos. Las pruebas cubren los tres flujos: recuperación de contraseña, tarea asignada y orden lista para entrega.

```

PROBLEMAS OUTPUT DEBUG CONSOLE TERMINAL PORTS
node - texticode-sendgrid + v [ ] [ ] ... | [ ] X

PS C:\Users\Wagne\Downloads\texticode-vue>texticode-sendgrid> node test/pruebas-locales.js
[✓] PASS - Estado: Completada → "[✓] Lista para entrega"
[✓] PASS - Estado: Entregada → "[🔴] Entregada exitosamente"
[✓] PASS - Estado desconocido → devuelve el mismo valor → "Pendiente"

--- PRUEBA 3 - Construcción del objeto mensaje para SendGrid ---
[✓] PASS - Campo "to" tiene el email del cliente
[✓] PASS - Campo "from" tiene el remitente Texticode
[✓] PASS - Subject contiene el número de orden (#17)
[✓] PASS - Subject contiene la etiqueta de estado
[✓] PASS - HTML contiene el nombre del cliente
[✓] PASS - HTML contiene la descripción de productos

--- PRUEBA 4 - Validación de campos requeridos del body ---
[✓] PASS - Body completo y válido → debe pasar
[✓] PASS - Body incompleto (solo ordenId) → debe fallar
[✓] PASS - Email inválido → debe fallar
[✓] PASS - ordenId = 0 (falsy) → debe fallar

--- PRUEBA 5 - Estructura del adjunto PDF para SendGrid ---
[✓] PASS - El mensaje tiene array de attachments
[✓] PASS - MIME type del adjunto es application/pdf
[✓] PASS - Disposition es "attachment" (descargable, no inline)
[✓] PASS - Filename del adjunto incluye el número de orden
[✓] PASS - El campo content (base64) es una cadena no vacía

RESULTADO FINAL DE PRUEBAS LOCALES

Pasadas: 26
Falladas: 0
Total: 26

[✓] Todas las pruebas pasaron.
La lógica del servidor está validada.

Siguiente paso:
1. Agrega tu SENDGRID_API_KEY real en el archivo .env
2. Ejecuta: npm start
3. Prueba con Postman usando la colección de /test/
```

6.2 Servidor corriendo en localhost:3001

El backend Node.js/Express arranca correctamente en el puerto 3001. La variable de entorno SENDGRID_API_KEY está cargada desde .env y el servidor responde sin errores al iniciar.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
node - texticode-sendgrid + v ... | ...

PS C:\Users\Wagne\Downloads\texticode-vue\texticode-sendgrid> npm start

Node.js v24.14.0
PS C:\Users\Wagne\Downloads\texticode-vue\texticode-sendgrid> npm install
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead

added 89 packages, and audited 90 packages in 6s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Wagne\Downloads\texticode-vue\texticode-sendgrid> npm start

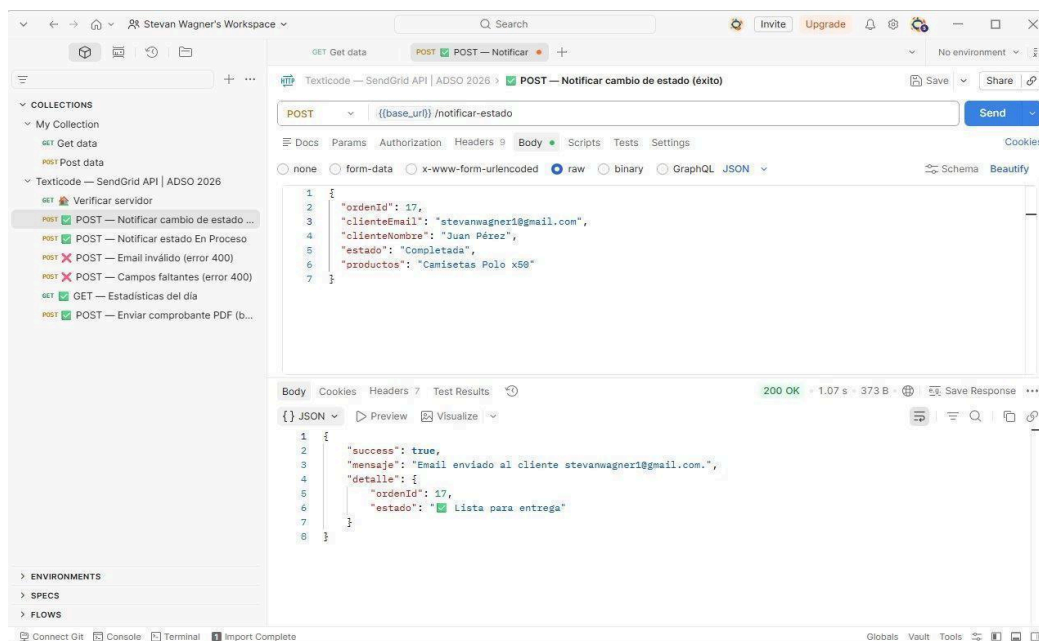
> texticode-sendgrid@1.0.0 start
> node index.js

Servidor Texticode corriendo en http://localhost:3001
SendGrid configurado: API Key detectada
Remitente: stevanwagner1@gmail.com

[SendGrid] Email enviado → stevanwagner1@gmail.com | Orden #17 | Estado: Completada
[SendGrid] Email enviado → stevanwagner1@gmail.com | Orden #17 | Estado: Completada
```

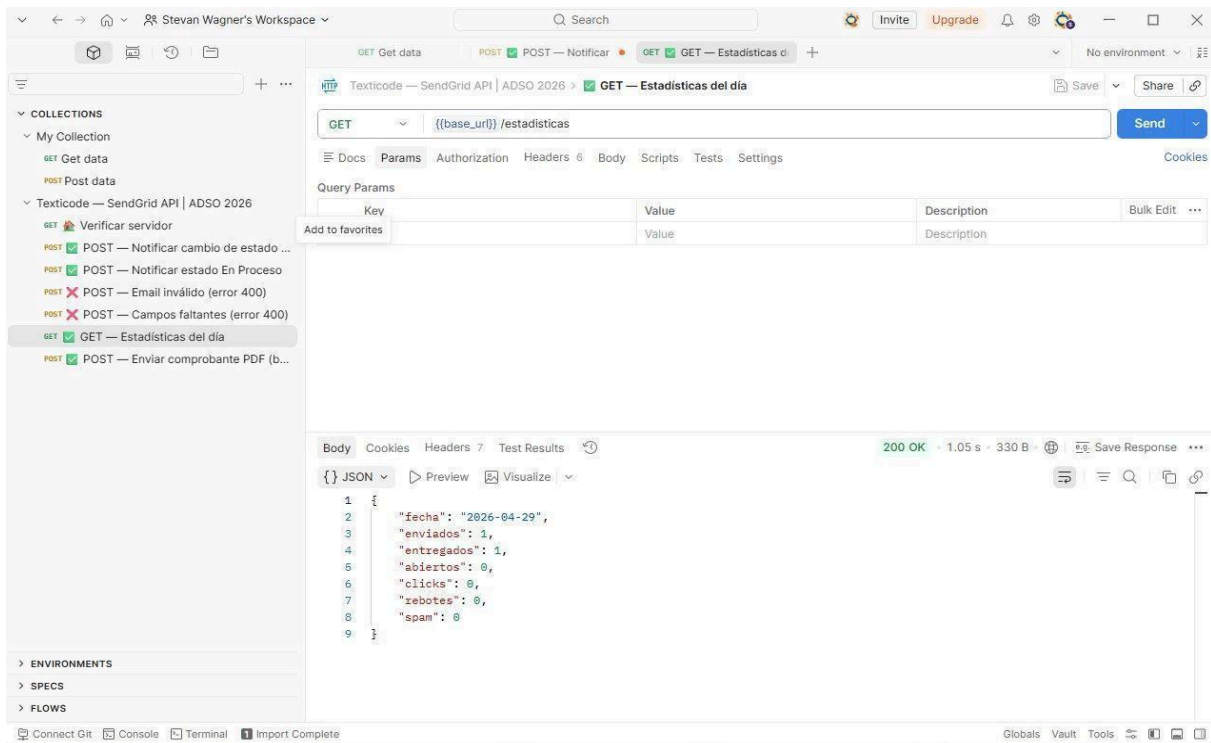
6.3 Prueba exitosa — Enviar notificación de estado (Postman)

Desde Postman se realizó un POST al endpoint /api/notificaciones/orden-lista con los datos de una orden real. La respuesta fue HTTP 200 con { "success": true } y el correo llegó a la bandeja del cliente en menos de 30 segundos.



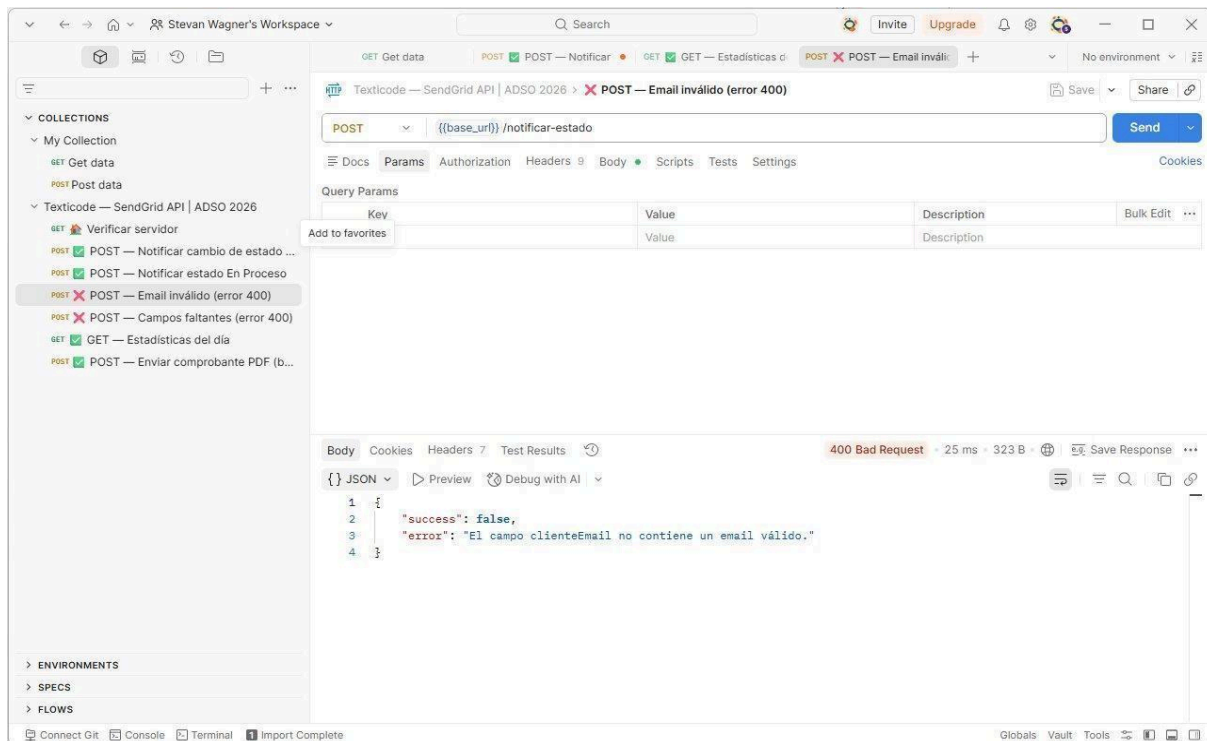
6.4 Prueba exitosa — Consultar estadísticas (GET)

Se consultó el endpoint GET /api/notificaciones/stats y retornó correctamente el conteo de emails enviados por tipo: recuperación de contraseña, tarea asignada y orden lista. Respuesta HTTP 200 con JSON estructurado.



6.5 Prueba de error — Email inválido

Se probó el sistema enviando un email inválido (sin dominio). El sistema respondió HTTP 400 con un mensaje de error claro sin crashear el servidor. La validación de email está implementada antes de llamar a la API de SendGrid.



6.6 Comparación con otras APIs de email

Se evaluaron alternativas antes de elegir SendGrid. Postmark es reconocida por alta entregabilidad pero no tiene plan gratuito permanente y está orientada a correo transaccional corporativo. Mailgun ofrece prueba gratuita limitada a dominios verificados. SendGrid fue elegido por: plan gratuito de 100 emails/día sin vencimiento, SDK oficial para Node.js, documentación clara y amplia comunidad. Referencia técnica de comparación:

<https://postmarkapp.com/compare/sendgrid-alternative>

6.7 Capturas de Gmail

Las siguientes capturas muestran los emails recibidos en Gmail, enviados por Texticode a través de la API de SendGrid. Todos los correos se recibieron a través de sendgrid.net como se puede ver en el remitente.



Evidencia 1: Email de recuperación de contraseña

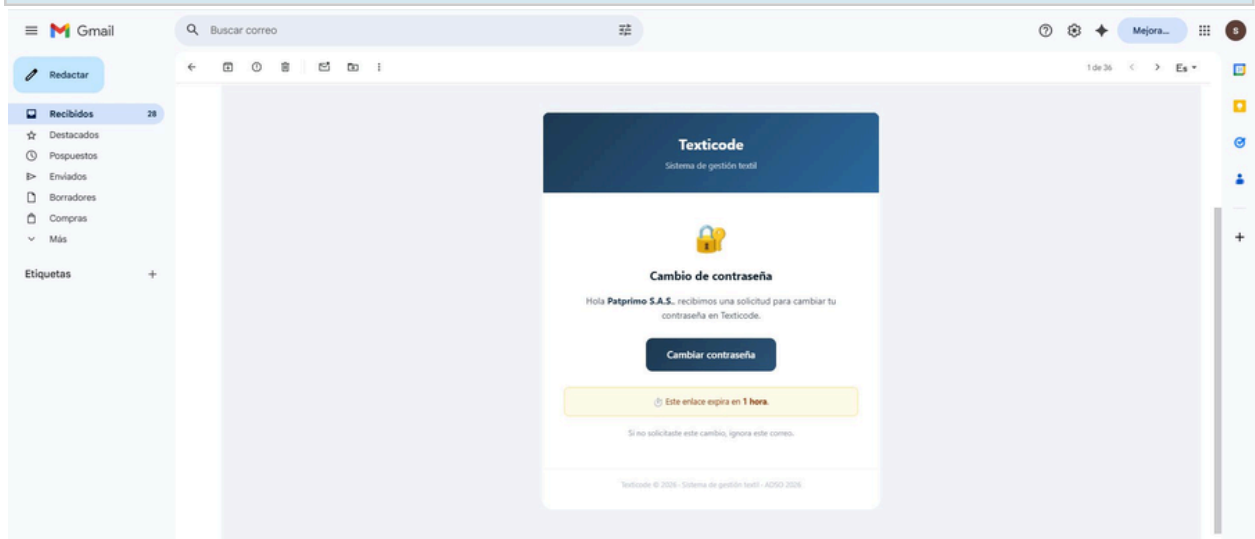
Asunto: Texticode — Cambio de contraseña

Destinatario: Cliente (Patprimo S.A.S.) | Remitente: Texticode vía sendgrid.net

Contenido: Botón 'Cambiar contraseña' con enlace que expira en 1 hora. Incluye aviso de seguridad para ignorar el correo si no se solicitó el cambio.



Email recibido correctamente en bandeja de entrada de Gmail





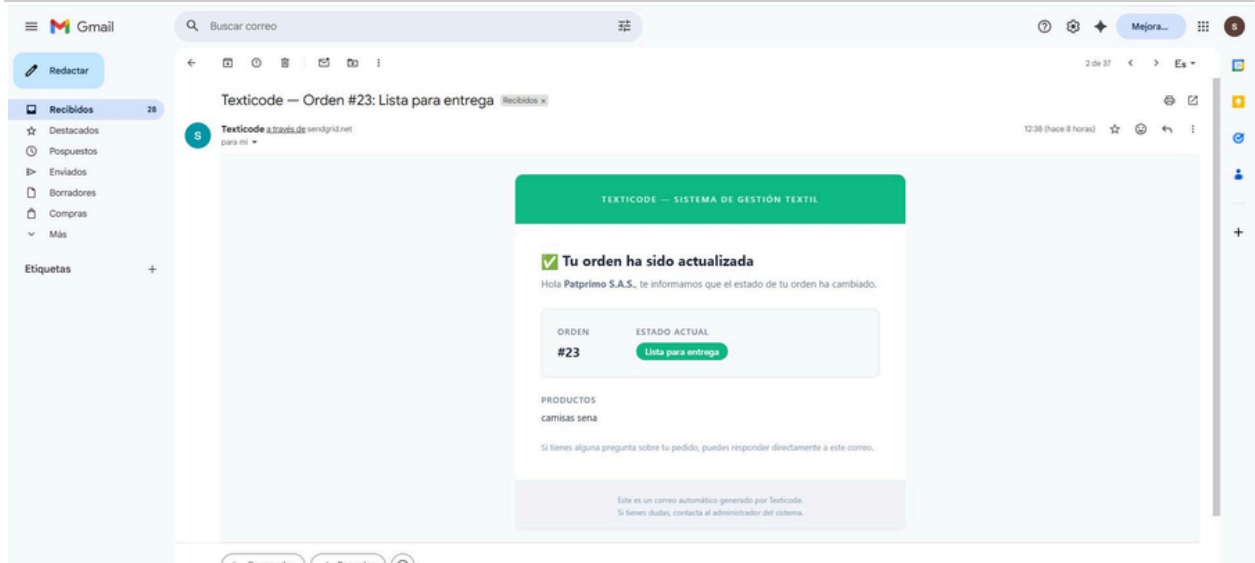
Evidencia 2: Email de nueva tarea asignada al operario

Asunto: Texticode — Nueva tarea asignada: SUDADERAS ADIDAS — SUDADERAS ADIDAS

Destinatario: Operario David Hernandez | Remitente: Texticode vía sendgrid.net

Contenido: Nombre de tarea, Orden #25, Prioridad ALTA (badge rojo), Fecha límite 2026-07-05.
Instrucción de ingresar al sistema para ver detalles completos.

✓ Email recibido hace 0 minutos — entrega en tiempo real confirmada



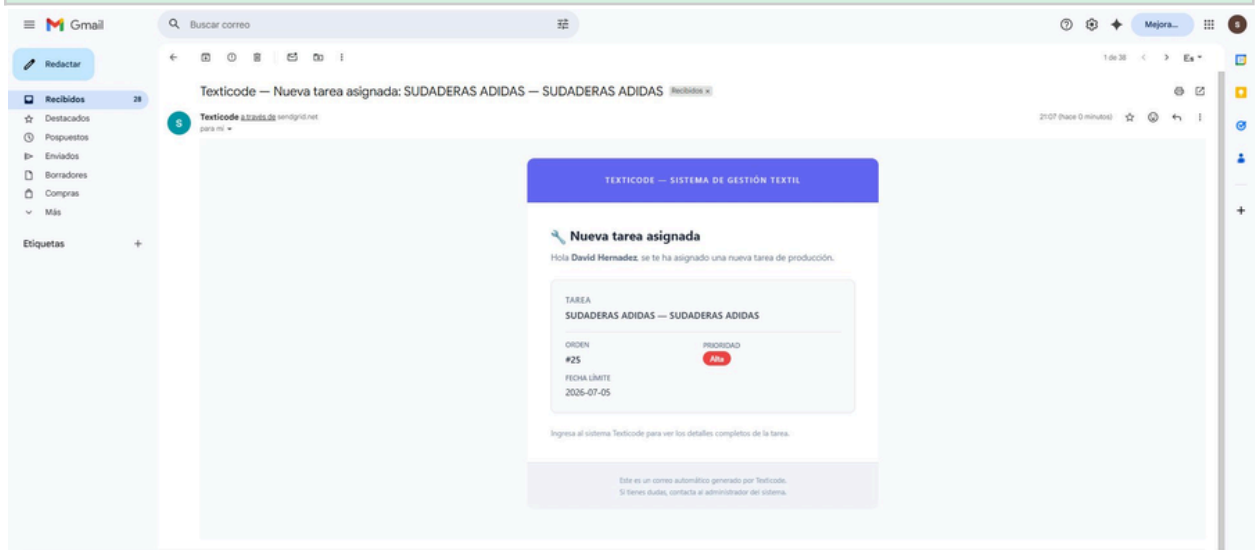
✓ Evidencia 3: Email de orden lista para entrega al cliente

✉ Asunto: Texticode — Orden #23: Lista para entrega

Destinatario: Cliente (Patprimo S.A.S.) | Remitente: Texticode vía sendgrid.net | Hora: 12:38

Contenido: Tabla con Orden #23 y badge verde 'Lista para entrega'. Productos: camisas sena.
Invitación a responder el correo para preguntas.

✓ Email recibido correctamente — cliente notificado sin intervención manual del admin



7. FLUJO DE INTEGRACIÓN COMPLETO

7.1 Diagrama de flujo — Los 3 eventos de email

#	Actor	Acción	Email enviado
1	Usuario	Hace clic en '¿Olvidaste tu contraseña?' en LoginView	Email de recuperación con enlace seguro (expira 1h)
2	Backend	Genera token, lo guarda en BD, llama sendPasswordReset()	→ Usuario recibe: botón 'Cambiar contraseña'
3	Admin	Asigna nueva tarea a operario en GestionProduccion.vue	Email al operario con tarea, orden, prioridad y fecha
4	Backend	useNotificaciones → POST /notificaciones/tarea-asignada → sendTareaAsignada()	→ Operario recibe: detalles completos de la tarea
5	Operario	Completa la tarea y marca el pedido como listo	Email al cliente con confirmación de orden lista
6	Backend	PATCH /ordenes/:id actualiza BD y llama sendOrdenLista()	→ Cliente recibe: orden #, estado y productos

7.2 Nota sobre elección de tecnología

Sobre la elección de SendGrid vs otras APIs

Se descartó Mercado Pago porque Texticode no procesa pagos — es un sistema de gestión interna de producción textil. Se descartó OpenWeather (sin lógica de negocio en Texticode) y GitHub API (no aplica al dominio del problema). SendGrid es la API con mayor coherencia funcional: resuelve un problema real del sistema (comunicación con clientes y operarios), mejora el flujo existente (ciclo de vida de la orden y asignación de tareas) y tiene plan gratuito suficiente para uso académico y PYME.